

AquaGuardian AI — Technical Report

GPIW 2026 Submission · Supporting Evidence · Track A (TRL 1–3) · Digitalisation, Automation and Smart Water Management

1. Overview and Scope

AquaGuardian AI is a closed-loop engineering platform that simulates, stress-tests, and validates a candidate water-infrastructure action before it is executed. This document describes the system as implemented in the submitted repository: its architecture, decision engine, bundled hardware/protocol simulator, interfaces, security posture, and deployment roadmap. It is a TRL 3 proof of concept — a working software prototype and reproducible simulation, not a field-deployed system. Companion documents: the *PoC Test Report* (automated test results and comparative figures) and the *Research Report* (methodology, related work, and threats to validity).

2. System Architecture

The system is organized into five layers, each mapping to a stage of the closed-loop decision cycle: **Sense** → **Analyze** → **Simulate** → **Stress Test** → **Optimize** → **Validate** → **Execute**. If a candidate plan fails Stress Test, it is rejected, automatically re-optimized, and re-tested before any further stage is attempted — no stage after Stress Test is reachable without a passing result.

Layer	Role	Implementation
L1 — Living Sensors	Water quality, pressure, flow, soil, plant bio-signal, and weather inputs	aquaguardian/scenarios.py (simulated sensor frames)
L2 — Green Box Edge AI	Local inference and fault detection, offline-capable	aquaguardian/detector.py
L3 — Living Digital Twin	Predicts flow, pressure, pump performance, leak propagation, fire risk	aquaguardian/digital_twin.py
L4 — Closed-Loop Engineering	Simulates, stress-tests, optimizes, and validates before execution	aquaguardian/engine.py, aquaguardian/policy.py
L5 — Autonomous Actions	Isolates leaks, reconfigures distribution, triggers fire prevention, notifies operators	aquaguardian/models.py (candidate action definitions)

2.1 Design Principles

- Edge-first and offline-capable — the decision path does not require a cloud round trip.
- No automatic action without simulation and validation.
- Fail-safe fallback under communication loss.
- Evidence attached to every decision (machine-readable decision trail).
- Human approval required for real deployments (see Section 6, control-mode progression).

3. Core Decision Engine

The engine (`aquaguardian/engine.py`) evaluates candidate actions for five simulated water-risk scenarios (`aquaguardian/scenarios.py`): pipeline leak, pump degradation, water contamination, drought stress, and wildfire risk. For each scenario, candidate actions are run through the stress-test matrix (`policy.py`) under four stress profiles — nominal, noisy sensors, high severity, and offline/degraded connectivity — and scored against constraints (safety risk, water loss, service disruption, contamination exposure, crop stress, fire risk). A plan that fails is rejected and the engine iterates toward a validated alternative before returning a decision.

The comparative evaluation module (`aquaguardian/comparison.py`) runs three decision strategies side by side — reactive baseline, detection-only AI, and the AquaGuardian closed loop — over the same scenario/stress-profile grid, producing per-scenario and aggregate metrics. Full results, methodology, and reproduction commands are in the companion PoC Test Report (Sections 4–5) and Research Report (Sections 4–5); this document does not repeat them in full.

4. Hardware and Protocol-Layer Integration

AquaGuardian's edge-AI and low-latency claims are backed by a bundled, independently developed architecture simulator, `hexa_tpu_rt_sim`, called live from `app/hardware_bridge.py` — not cited as a static specification. Four sub-layers are exercised on every call to `GET /api/hardware/reflex`:

Sub-layer	Module	Function
Reflex kernel	<code>reflex/reflex_kernel.py</code> , <code>reflex/domains/water.py</code>	Bounded-latency decide-and-act path with a hard sub-1ms deadline; a water-hammer/pipe-burst trigger. The same kernel class is reused, unmodified, for an aviation domain.
Edge → MEC network	<code>network/urllc_link.py</code>	Statistical model of the edge-to-MEC hop as a 5G NR URLLC link, calibrated to a 10ms SLA.
MEC platform & cognitive layer	<code>mec/mec_platform.py</code> , <code>mec/domains/water_cognitive_engine.py</code>	ETSI MEC-shaped host/platform/app registration; a bio-inspired decision stack (Bat Forecaster for remaining-useful-life projection, Squid Controller for weighted action selection, a Hermit-Crab-style hard veto against water-hammer-risk actions).
oneM2M middleware	<code>middleware/onem2m.py</code>	A TS-0001/TS-0004-shaped resource tree (MN-CSE, ADN-AE roles) with real CREATE/RETRIEVE primitive handling for edge-node registration and telemetry — an in-memory simulation of the standard's architecture, not a live wire-protocol binding.

Representative results from a live run (seed=0, 200 samples): reflex-kernel max decision latency 0.00003ms with 0 deadline misses against a 1ms deadline; network p99 latency 4.23ms against a 10ms SLA; MEC layer selects a staged valve shutdown and vetoes an emergency full shutdown on a worsening leak (severity 0.8); oneM2M AE registration, event CREATE, and an explicit RETRIEVE primitive all return success, with 5 requests logged. Full detail and reproduction commands: PoC Test Report, Section 5.

Scope note: these are cycle-accurate deadline checks and a statistical network model running in software, not measurements from real embedded hardware or a real radio. Three tests in `hexa_tpu_rt_sim/tests/test_middleware_mec.py` that require a live external oneM2M CSE server are skipped in this environment; they test the HTTP binding layer specifically, not the resource-tree logic exercised above.

5. System Interfaces

Endpoint	Method	Purpose
/api/scenarios	GET	Metadata for the scenario picker and digital-twin schematic (id, tag, name, location, twin-focus nodes).
/api/run/{scenario_id}	GET (SSE)	Streams the closed-loop cycle stage by stage as it executes server-side, including the stress-test failure → re-optimize → re-test loop, ending in a completion event with final metrics.
/api/comparison/{scenario_id}	GET	Per-scenario reactive vs. detection-only vs. closed-loop comparison, generated by aquaguardian/comparison.py.
/api/hardware/reflex	GET	Runs the hexa_tpu_rt_sim reflex/network/MEC/oneM2M benchmark live and returns the results (Section 4).

SSE event types emitted by /api/run/{id}: stage_start, stress_fail, reoptimize_start/reoptimize_done, stress_retry_start, stage_done, and complete — the frontend console renders each event as it arrives, so the pipeline visualization reflects server-side sequencing rather than a client-side script.

6. Security Architecture and Threat Model

Assets: water availability and quality; pumps, valves, and irrigation actuators; sensor integrity; operator accounts; network topology and operational data; decision and audit records.

Trust boundaries: (1) sensor/edge-device boundary; (2) edge-to-platform communication boundary; (3) human operator and approval boundary; (4) platform-to-SCADA/actuator boundary; (5) software update and configuration boundary.

Highest-consequence failure: a false or manipulated decision that causes unsafe valve, pump, or irrigation behavior. The production architecture must therefore keep deterministic safety constraints outside any probabilistic model and require bounded, authenticated commands.

PoC exclusions (intentional at TRL 3): the current repository does not include real credentials, production SCADA drivers, remote actuator control, or a production identity provider. It has no user identity system, no actuator credentials, and no direct SCADA connection. CORS defaults to local development origins; production deployments must set an explicit allow-list.

6.1 Control-Mode Progression

Mode	Description
1. Replay	Recorded data only — no live sensors, no actuation.
2. Shadow	Live data ingested; recommendations only, no actuator commands.
3. Supervised	Operator approves each action before it executes.
4. Bounded autonomy	Only pre-approved, low-risk actions execute automatically.

Production controls planned for later phases span four categories: identity and access (mutual TLS for edge devices, OAuth2/OIDC for human users, role-based access control, short-lived credentials); command safety (signed commands, sequence numbers, allow-listed actions, two-person approval for high-impact operations, local hard safety limits independent of AI decisions); data integrity (replay protection, sensor-quality scoring, encrypted transport/storage, immutable audit records); and edge resilience (secure boot, signed firmware, watchdog

monitoring, store-and-forward operation, signed OTA updates with rollback). None of these are implemented in the current TRL 3 package. No production-readiness or security claim should be made before an independent security review and site-specific hazard analysis.

7. Deployment Roadmap

Phase	Target	Duration	Key activities
0 — Current submission	TRL 3	—	Deterministic simulation, reproducible baselines, API/UI/tests/Docker; no field-control claim.
1 — Laboratory validation	TRL 4	3–6 months	Connect real sensors and a small pump/valve test loop; replay fault traces; calibrate; measure latency, false alarms, model error; complete security/hazard reviews.
2 — Shadow-mode pilot	TRL 5	6–9 months	Deploy beside an operating farm/irrigation operation; ingest live data without control; compare recommendations to operator actions; quantify impact.
3 — Supervised pilot	TRL 6	—	Enable operator-approved low-risk actions; integrate identity/audit/command signing; independent cybersecurity and safety review.
4 — Commercial deployment	—	—	Packaged site templates, SLAs, device lifecycle and secure-update process, integration partner channel.

8. Known Technical Limitations

- The hydraulic, agronomic, and fire models used in simulation are simplified, not calibrated against a real hydraulic network, weather record, or fire-spread model.
- Sensor calibration, actuator reliability, local regulation, and human-in-the-loop procedures all require field validation before any deployment claim can be made.
- Results should be used to compare software strategies inside this PoC only — not treated as guaranteed savings, safety performance, or deployment-ready control parameters.
- `hexa_tpu_rt_sim`'s edge/network/MEC numbers are software simulations, not measurements from real embedded hardware or a real radio (Section 4).

9. Provenance and Reuse

The submitting developer owns the AquaGuardian source code and the previously developed project components and engineering patterns reused in this submission. No third-party proprietary source code is knowingly included. Reused engineering lineage: **Black Dragon Studio** (simulation-first validation workflow and testing discipline), **Autex** (reliability, confidence, and fail-safe decision-evidence patterns), **IrrigaFire** (water-stress, irrigation, and wildfire-prevention domain concepts), and **Green Box / Nomad Sentinel** (edge-first, offline-resilient architecture patterns) — all created by the submitting developer. Open-source Python dependencies are listed in `requirements.txt` and remain subject to their respective licenses.